

Benchmark Report

Overview

This project benchmarks the performance of multiple JSON processing solutions under different workloads and dataset sizes. The primary goal is to compare execution speed across different implementations.

The benchmark focuses on common JSON processing operations such as filtering, mapping, and length calculation on generated datasets of varying sizes and nesting depths.

Compared Software

Software	Description
Custom Java (Single-threaded)	A custom JSON processor implemented in Java using a single-threaded execution model
Custom Java (Multi-threaded)	A multi-threaded variant of the Java implementation
Custom Python	A Python implementation generated by GitHub Copilot based on my Java implementation. ▼ [prompt] Create a similar json processor in Python to what I created in <code>json_bench_java</code>, with the same commands, but only single-threaded, not multiple engines. The project has a base in <code>json_bench_python</code>. Make it high quality and well structured.
jq	The widely used command-line JSON processor

Motivation Behind Software Selection

The selected software solutions represent different approaches to JSON processing:

- The custom Java implementations allow direct comparison between single-threaded and multi-threaded processing strategies.
- The Python implementation provides insight into the performance characteristics of an interpreted language compared to Java.
- jq serves as an established and highly optimized baseline commonly used for command-line JSON processing tasks.

Software

Operating System	Windows 11 Home
OS Build	25H2
Java Version	21.0.2
Python Version	3.12.0
jq Version	1.8.1

Hardware

CPU	AMD R7 5825U (8C16T @ 2GHz; Boost 4.5GHz)
CPU Cache	L1: 512 KB; L2: 4 MB; L3: 16 MB
Memory	24 GB DDR4 3200 MT/s
Storage	Samsung NVMe SSD

Benchmark Environment

All 3 implementations are benchmarked using `hyperfine` with 3 warmup- and 5 benchmark runs.

Dataset Generation

Synthetic JSON datasets are generated programmatically using a custom Python generator script.

The datasets vary in:

- File size
- Record count
- Nesting depth
- Structural complexity

Dataset Sizes

Dataset	Record Count	Approximate Size @ no nesting
Small	10k records	0.8 MB
Medium	100k records	8.5 MB
Large	1M records	87 MB
Very Large	5M records	440 MB
Extra Large	10M records	890 MB

Test Cases

The following operations are benchmarked:

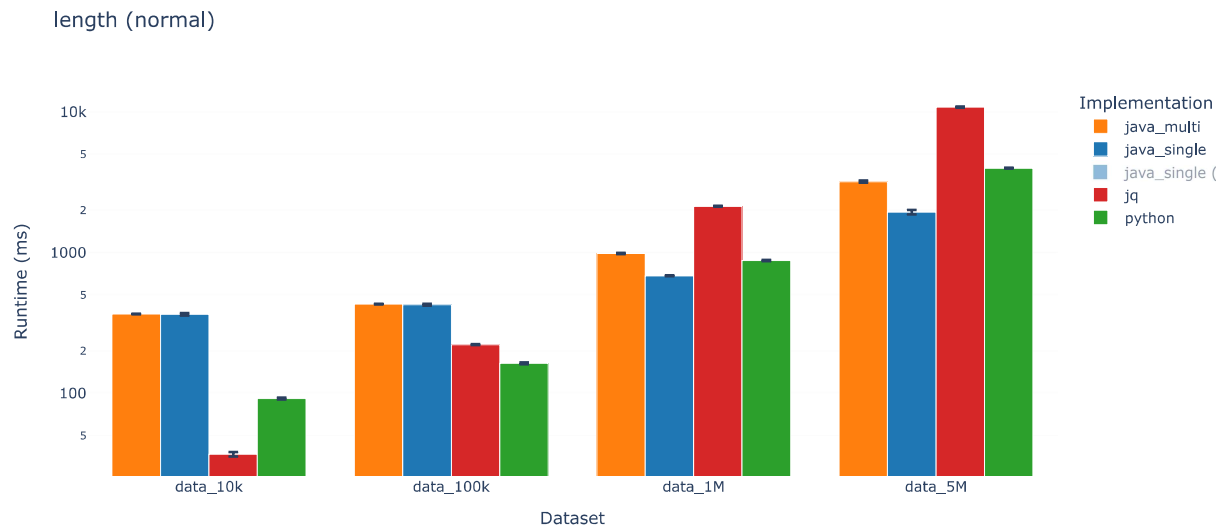
Operation	Description
length	simply counts the number of records
filter	<code>id == 5555 age < 23</code>
map	applies <code>^6</code> to every numerical value.

Benchmark results

The charts use **logarithmic** y axes.

▼ length

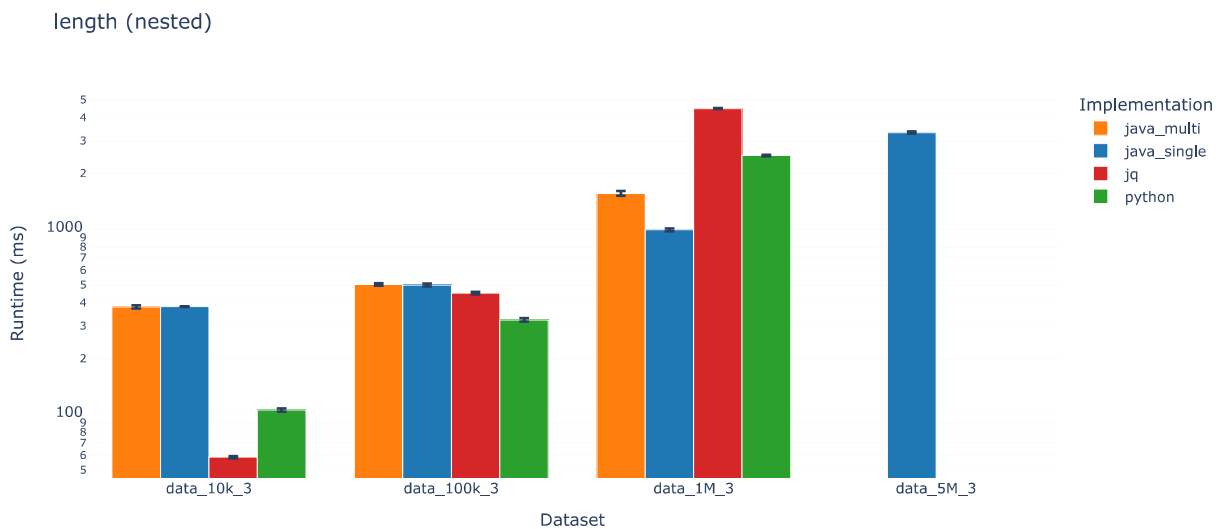
▼ normal



This is the simplest operation. On the small file, jq is much faster, than the custom implementations, which is no surprise, since it's implemented in C/Haskell and is well optimized. It's 10 times faster than the Java implementations, which are slowed down by the JVM startup. Python is faster on the small file than Java, probably because its startup is faster, than JVM, and the operation is not difficult/long enough to make a difference.

On larger files jq interestingly falls behind, and the single-core implementation of Java starts to take lead, with the multi-threaded catching up, since the thread overhead is still dominating here. Python is also slowly falling behind as the file sizes increase.

▼ nested

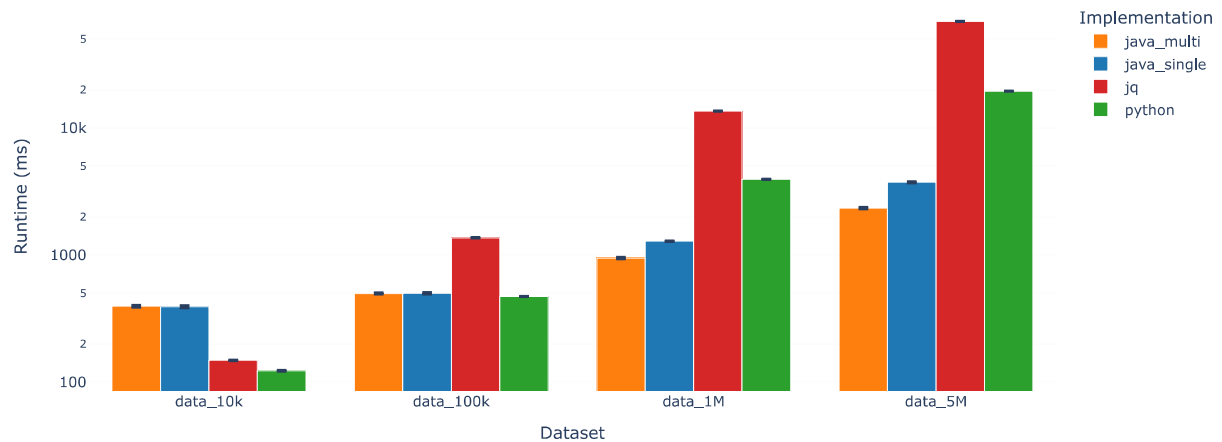


Here, using the nested structures, we can see similar differences between the languages. We see pretty small differences on smaller files and on the larger file about 50% runtime increase in Java, and 100% runtime increase in the other two compared to the non-nested json files.

▼ filter_age

▼ normal

filter_age (normal)



This operation is already more difficult, than the length one. The small file size still produces a similar result, but as the size increases, we see different results.

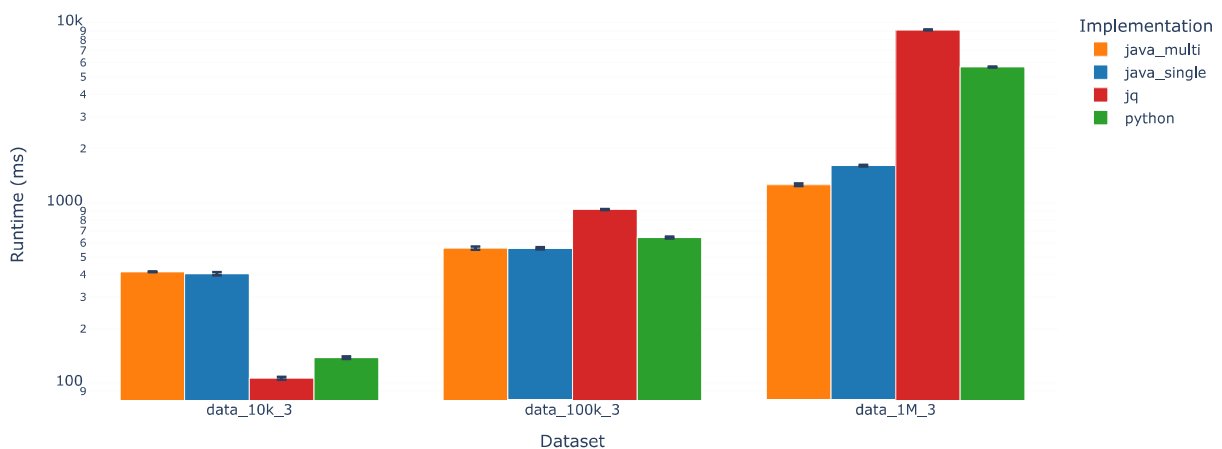
JQ is falling behind by a lot, in my opinion because it's much more versatile, than the custom software.

Especially as the file sizes increase, we see the multi-threaded implementation starts outperforming the other ones. It takes time to split and merge the tasks, manage the threads, but with more operations to complete, multiple threads could still finish the task faster, than just 1 thread working on them. On the file with 5 million records, it's already 50% faster, than the single-threaded equivalent.

Here we can start noticing the difference between a compiled, and an interpreted language as well. At 1 million records, the Java implementation is already almost 4 times faster, and this difference increases quickly with size.

▼ nested

filter_age (nested)

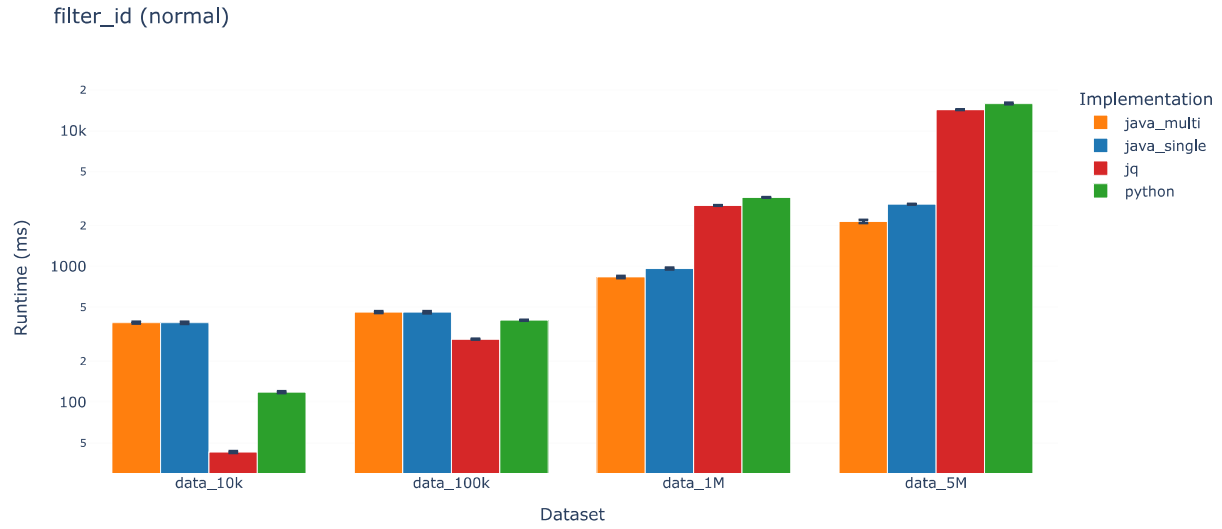


The differences between the different implementations are very similar to the non-nested ones again.

Runtimes are slower, but only by ~30% at 1 million records.

▼ filter_id

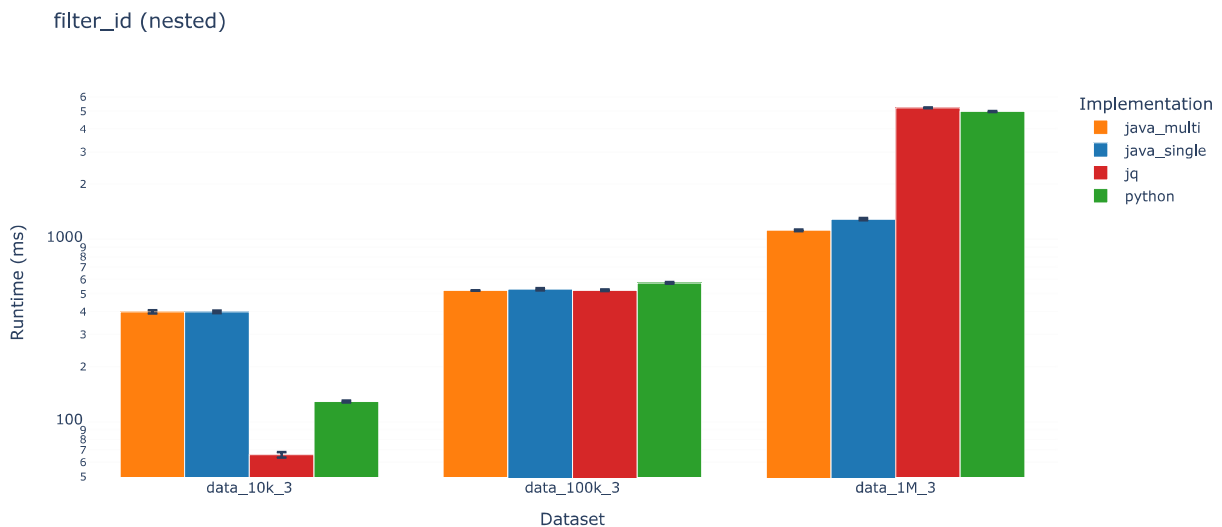
▼ normal



Here we have a simpler operation, than the previous one. This one searches for a specific ID, whereas the previous one selected every record with an age < 23. This makes the results a bit more similar to the length results.

On the small file, jq is again a lot faster, than the custom implementations. On larger files, the Java implementations start to take the lead, and interestingly the multi-threaded one is already the fastest at 1 million records. At 5 million records, the differences already become very large, with the multi-threaded Java being ~8 times faster than the Python.

▼ nested

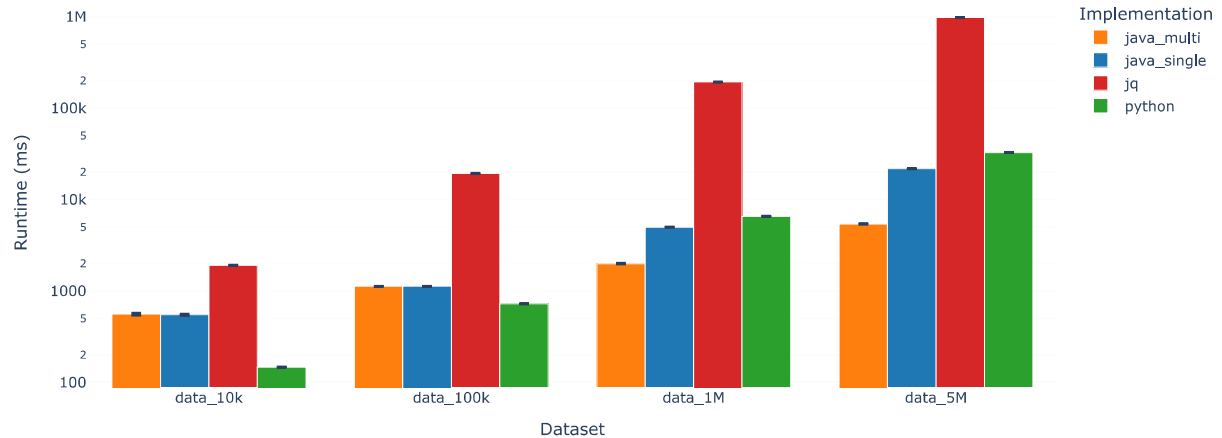


With the nested structure, we see very similar differences between the languages again, with a noticeable runtime increase, especially for JQ and Python on larger files.

▼ map_pow6

▼ normal

map_pow6 (normal)



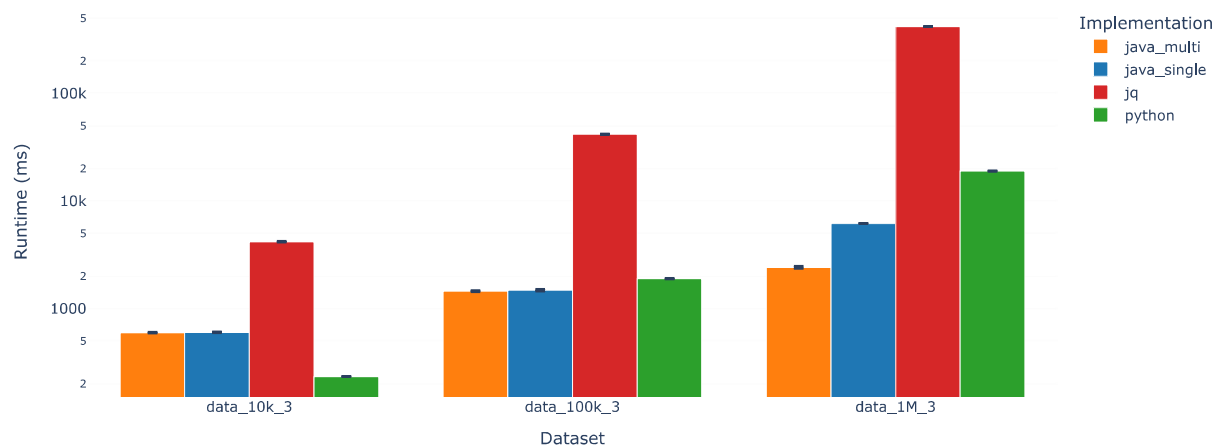
This is the most computationally intensive operation, and we notice the multi-threaded implementation's advantage even more quickly. On larger files the differences become very significant, with the multi-threaded being almost 4 times as fast as the single-threaded one at 5 million records.

JQ struggled from the start compared to my implementations, because I could optimize them for this specific task, whereas JQ is much more versatile. There are very significant differences, that are only getting bigger as the file sizes increase.

I found it interesting, that at 100k records, the Python implementation was still faster, than the Java ones. This changed as the files got larger, with Python being 50% slower compared to Java at 5 million records.

▼ nested

map_pow6 (nested)



The nested structure shows similar results, as all the previous ones again. Here, Python was already slower at 100k records, than the Java ones. The Python implementation saw a larger runtime increase, than the Java implementations.

Conclusions

Generally Java outperforms Python and jq on the larger datasets, with jq being the fastest on small datasets, except with the last operation. Python is competitive on small files, where the JVM startup time matters more, than the processing time. On medium datasets, Python tends to be similar to Java. With the map operation, which is computationally heavy, jq's performance degraded exponentially as dataset sizes increased.

Nested datasets consistently increased runtime for all implementations, but the impact was uneven:

- Java shows moderate overhead, approximately 10-40% increase depending on operation.
- Python shows larger degradation, especially in map operation, with a 3× runtime increase on larger datasets.
- jq's runtime increased by roughly 2×

The benchmarks show that a multi-threaded implementation provides clear benefit on larger datasets, but on smaller ones, the single-threaded is equal or faster due to thread overhead. Compute-heavy operations show larger performance differences in favor of the multi-threaded implementation.

Key takeaways:

- Java showed the most consistent scaling across the different workloads and dataset sizes.
- Python is competent under small to medium workloads, but falls behind quickly as datasets and operation difficulty increase.
- jq is very good for lightweight transformations, but it's not viable for large-scale operations.
- Nesting provided interesting, non-linear performance degradations.